In [ ]:
```
# OptiCrop: Smart Agricultural Production Optimization Engine
### AI/ML Track |Team Lead: Geethika Konduru | Team Meambers   Ajjarapu Sowmya,Jada sravani,Mohammad Karishma,Neha Neeharika Nallagondla
**Dataset:** https://www.kaggle.com/datasets/chitrakumari25/smart-agricultural-production-optimizing-engine
```

In [ ]:
```
---
## Epic 2: Data Collection and Analysis
---
```

In [ ]:
```
### Task 1: Download the Dataset
- **Source:** Kaggle – Smart Agricultural Production Optimizing Engine
- **Link:** https://www.kaggle.com/datasets/chitrakumari25/smart-agricultural-production-optimizing-engine
- **File:** Crop_recommendation.csv | **Records:** 2200 | **Features:** N, P, K, temperature, humidity, pH, rainfall, label
```

In [ ]:
```
### Task 2: Importing the Libraries
Required libraries for data analysis, ML operations, and visualization are imported below.
```

In [2]:
```
import pickle
import sqlite3
import numpy as np
import pandas as pd

from flask import (
    Flask,
    render_template,
    request,
    redirect,
    url_for,
    session,
    flash
)

from werkzeug.security import (
    generate_password_hash,
    check_password_hash
)

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

In [ ]:
```
### Task 3: Read the Dataset
The dataset is loaded using `read_csv()` and explored using `head()`, `info()`, and `describe()`.
```

In [9]:
```
import pandas as pd

data = pd.read_csv(r"C:\Users\HP-PC\Downloads\evaluatingmodelperformanceandsavingthebestmodel\Crop_recommendation.csv")
df = data.copy()

print("Dataset Shape:", df.shape)
df.head()
```

```
Dataset Shape: (2200, 8)
```

Out[9]:

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

In [10]:
```
data.shape
```

Out[10]:
```
(2200, 8)
```

In [11]:
```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   N            2200 non-null   int64  
 1   P            2200 non-null   int64  
 2   K            2200 non-null   int64  
 3   temperature  2200 non-null   float64
 4   humidity     2200 non-null   float64
 5   ph           2200 non-null   float64
 6   rainfall     2200 non-null   float64
 7   label        2200 non-null   str    
dtypes: float64(4), int64(3), str(1)
memory usage: 153.0 KB
```

In [12]:
```
print('Unique Crops:', df['label'].nunique())
print('Crop List:', sorted(df['label'].unique()))
```

```
Unique Crops: 22
Crop List: ['apple', 'banana', 'blackgram', 'chickpea', 'coconut', 'coffee', 'cotton', 'grapes', 'jute', 'kidneybeans', 'lentil', 'maize',
'mango', 'mothbeans', 'mungbean', 'muskmelon', 'orange', 'papaya', 'pigeonpeas', 'pomegranate', 'rice', 'watermelon']
```

In [13]:
```
df.describe()
```

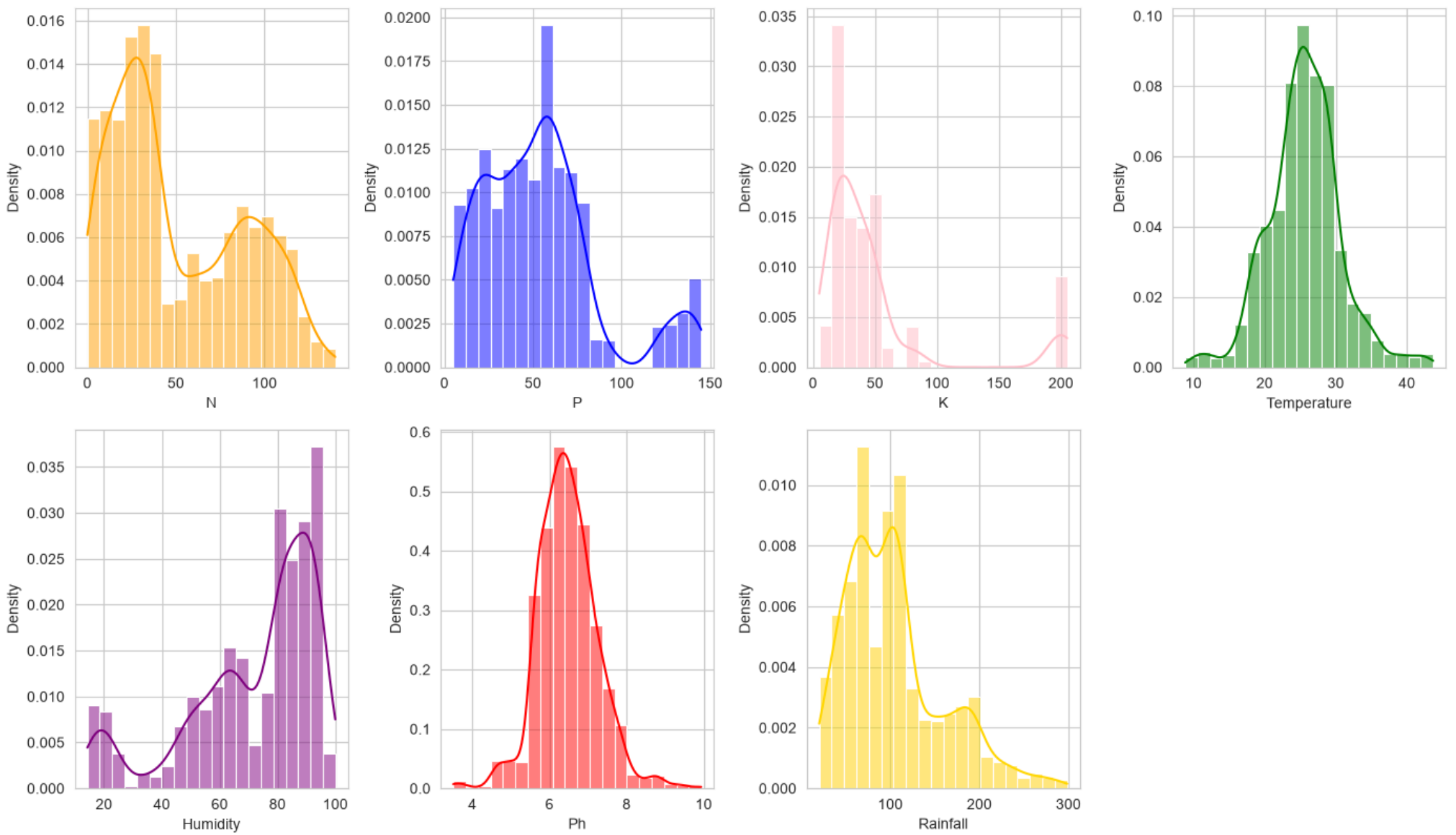
Out[13]:

	N	P	K	temperature	humidity	ph	rainfall
count	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000
mean	50.551818	53.362727	48.149091	25.616244	71.481779	6.469480	103.463655
std	36.917334	32.985883	50.647931	5.063749	22.263812	0.773938	54.958389
min	0.000000	5.000000	5.000000	8.825675	14.258040	3.504752	20.211267
25%	21.000000	28.000000	20.000000	22.769375	60.261953	5.971693	64.551686
50%	37.000000	51.000000	32.000000	25.598693	80.473146	6.425045	94.867624
75%	84.250000	68.000000	49.000000	28.561654	89.948771	6.923643	124.267508
max	140.000000	145.000000	205.000000	43.675493	99.981876	9.935091	298.560117

In [ ]:
```
### Task 4: Univariate Analysis
Distplot and countplot are used to analyze individual feature distributions.
```

In [16]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
df = pd.read_csv(r"C:\Users\HP-PC\Downloads\evaluatingmodelperformanceandsavingthebestmodel\Crop_recommendation.csv")

# Set plot style
sns.set_style("whitegrid")

# Features to analyze
features = ['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']
colors = ['orange', 'blue', 'pink', 'green', 'purple', 'red', 'gold']

# Create figure
plt.figure(figsize=(14, 9))
```

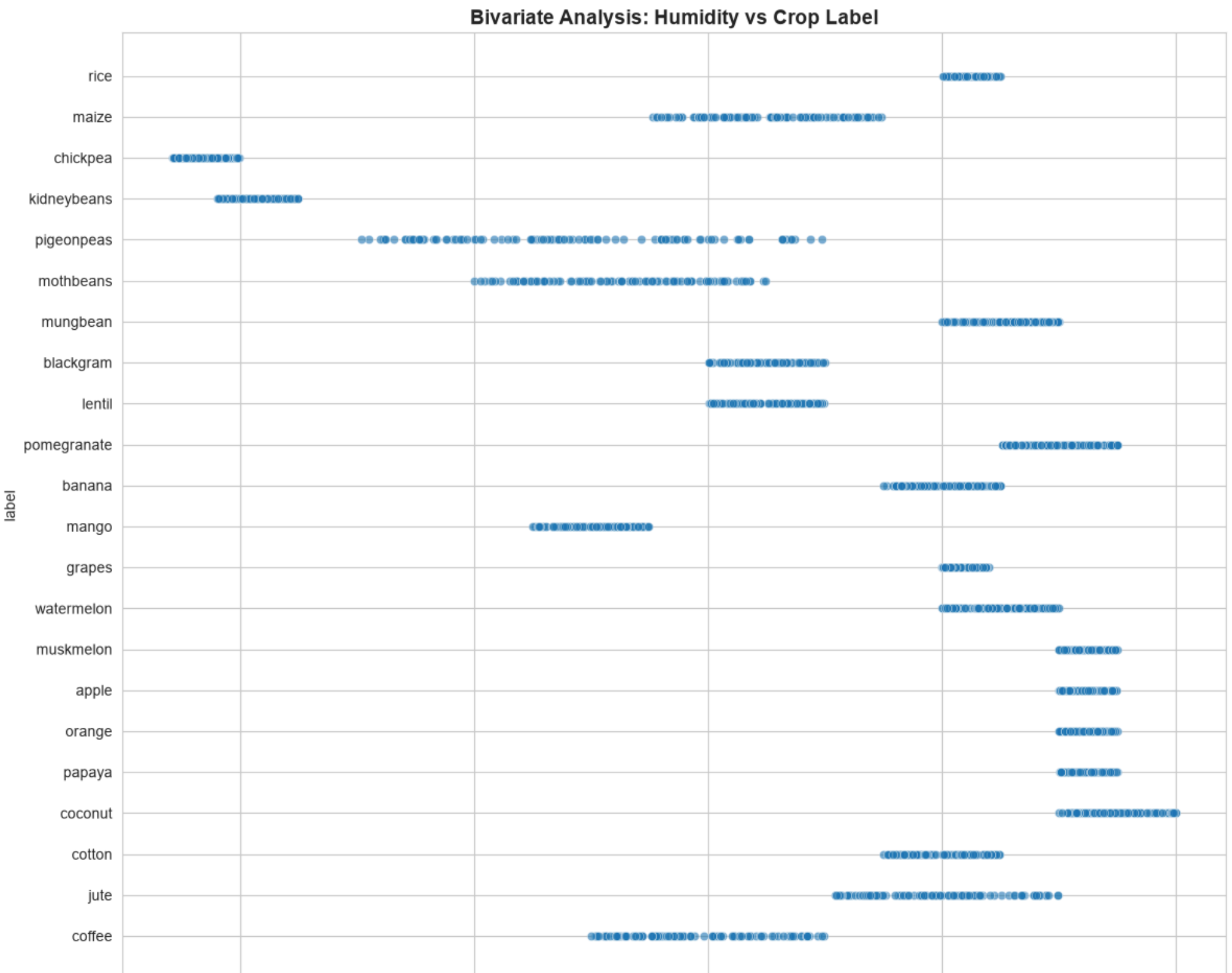
```
plt.suptitle("Distribution of Agricultural Conditions", fontsize=20, fontweight='bold')

# Plot histograms
for i, feature in enumerate(features):
    plt.subplot(2, 4, i + 1)

    sns.histplot(
        data=df,
        x=feature,
        bins=20,
        kde=True,
        color=colors[i],
        stat='density',
        edgecolor='white'
    )

    plt.xlabel(feature.capitalize(), fontsize=10)
    plt.ylabel("Density", fontsize=10)

# Adjust layout
plt.tight_layout(rect=[0, 0, 1, 0.95])

# Save the figure
plt.savefig("univariate_analysis.png", dpi=300)

# Display the figure
plt.show()
```

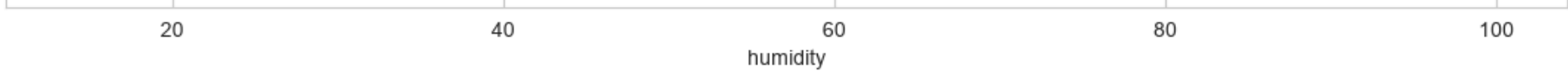


In [ ]:
```
### Task 5: Bivariate Analysis
Scatter plots are used to identify relationships between two agricultural features.
```

In [17]:
```
plt.figure(figsize=(12,10))
sns.scatterplot(x=df['humidity'], y=df['label'], alpha=0.6)
plt.title('Bivariate Analysis: Humidity vs Crop Label', fontsize=14, fontweight='bold')
plt.xlabel('humidity')
```

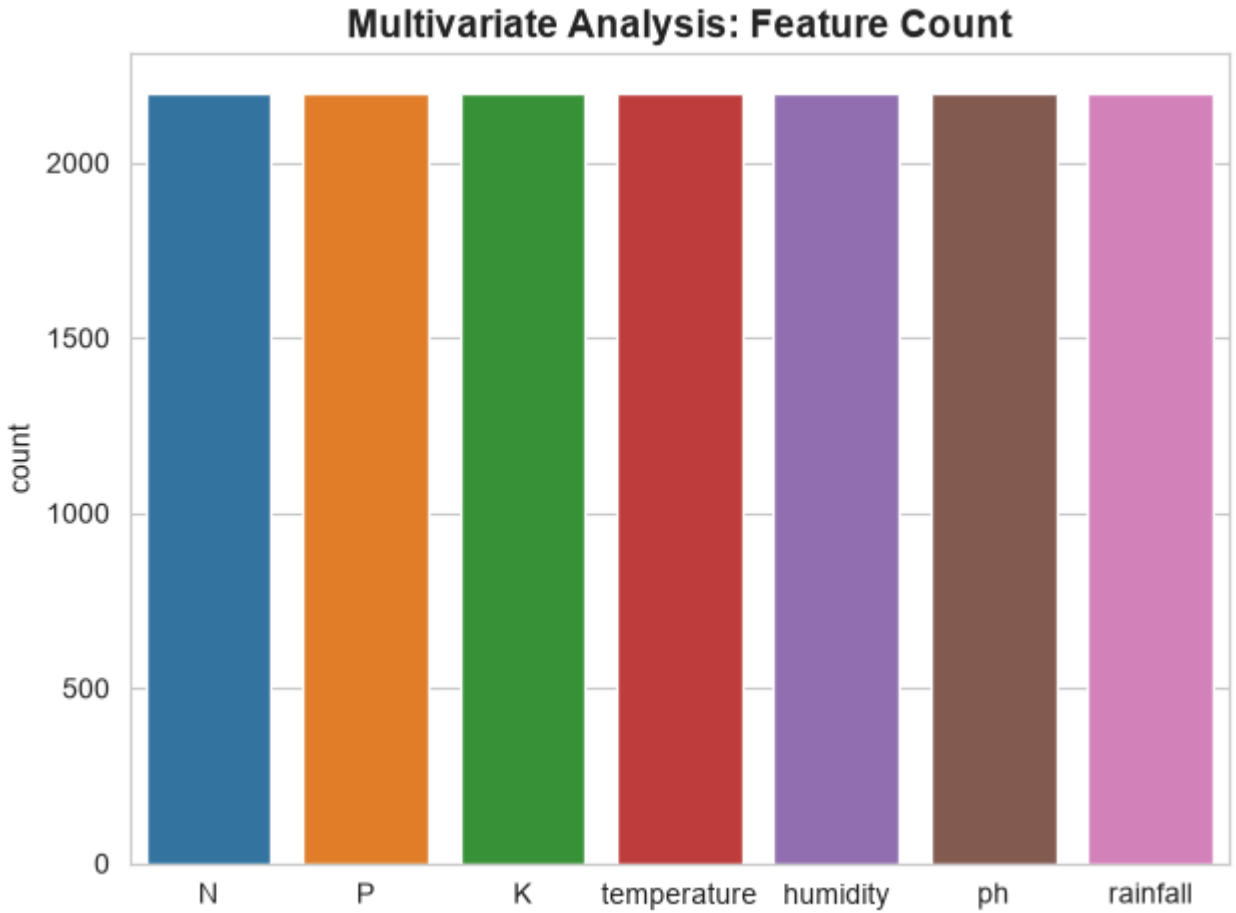
```
plt.ylabel('label')
plt.tight_layout()
plt.show()
```


Bivariate Analysis: Humidity vs Crop Label
label
rice
maize
chickpea
kidneybeans
pigeonpeas
mothbeans
mungbean
blackgram
lentil
pomegranate
banana
mango
grapes
watermelon
muskmelon
apple
orange
papaya
coconut
cotton
jute
coffee



In [ ]:
```
### Task 6: Multivariate Analysis
Countplot and correlation heatmap are used to study multiple feature relationships simultaneously.
```

In [18]:
```
sns.countplot(df)
plt.title('Multivariate Analysis: Feature Count', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



In [ ]:
```
---
## Epic 3: Data Pre-Processing
---
```

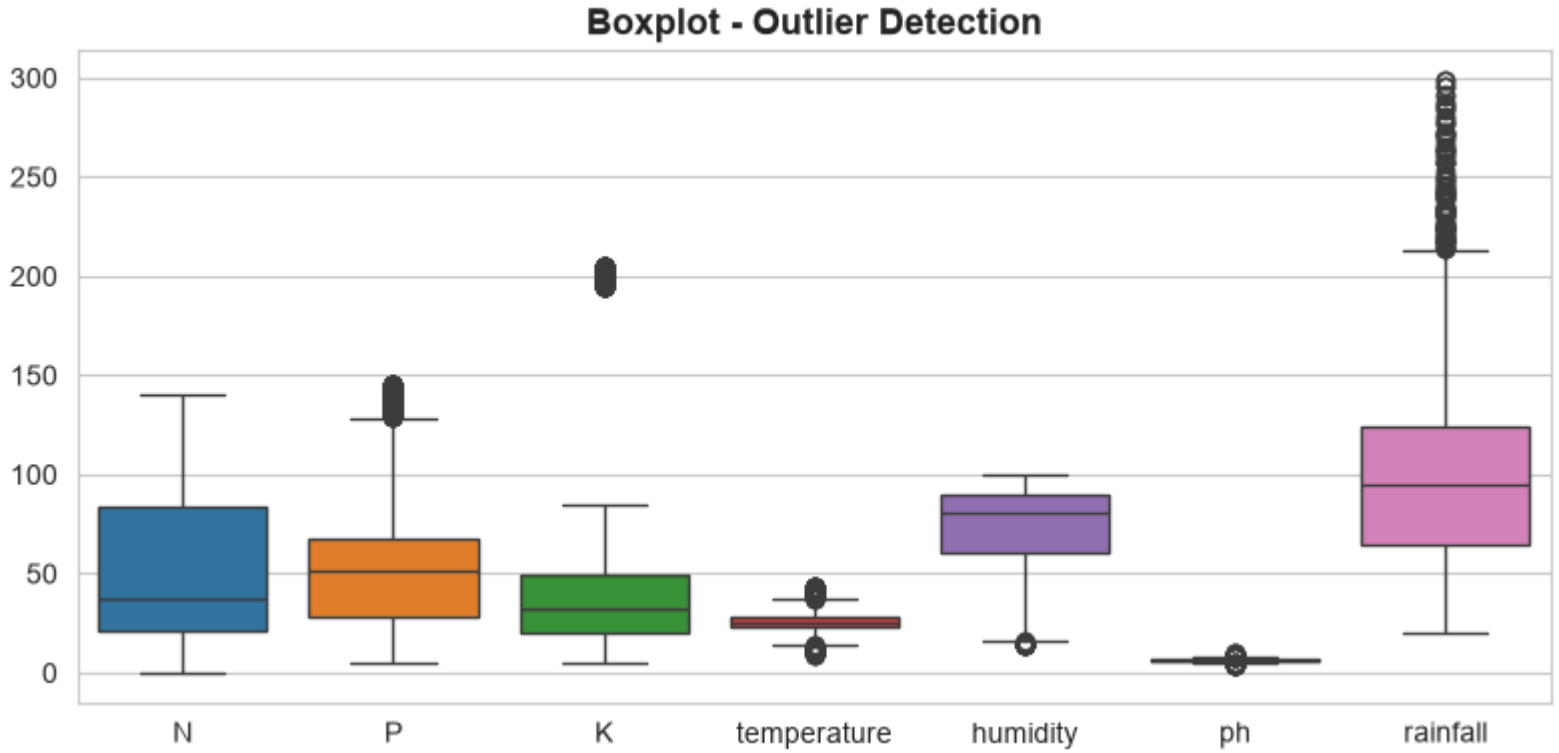
In [ ]:
```
### Task 1: Checking for Null Values
Using `df.shape`, `df.info()`, and `df.isnull().sum()` to check dataset structure and missing values.
```

In [19]:
```
df.isnull().sum()
```

Out[19]:
```
N              0
P              0
K              0
temperature    0
humidity       0
ph             0
rainfall       0
label          0
dtype: int64
```

In [ ]:
```
### Task 2: Handling Outliers
Boxplot is used to visualize outliers. IQR method is applied to treat outliers in the phosphorous feature.
```

In [20]:
```
plt.figure(figsize=(8,4))
sns.boxplot(df)
plt.title('Boxplot - Outlier Detection', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



In [ ]:
```
### Task 3: Extracting Seasonal Crops
Crops are grouped based on seasonal environmental conditions – Summer, Winter, and Rainy.
```

In [21]:
```
print('Summer crops')
print(df[(df['temperature']>30) & (df['humidity']>50)]['label'].unique())
print('-----------------------------------------------')
print('Winter crops')
print(df[(df['temperature']<20) & (df['humidity']>30)]['label'].unique())
print('-----------------------------------------------')
print('Rainy crops')
print(df[(df['rainfall']>200) & (df['humidity']>50)]['label'].unique())
print('-----------------------------------------------')
```

```
Summer crops
<ArrowStringArray>
['pigeonpeas', 'mothbeans', 'blackgram', 'mango', 'grapes', 'orange',
 'papaya']
Length: 7, dtype: str
--------------------------------------------------
Winter crops
<ArrowStringArray>
['maize', 'pigeonpeas', 'lentil', 'pomegranate', 'grapes', 'orange']
Length: 6, dtype: str
--------------------------------------------------
Rainy crops
<ArrowStringArray>
['rice', 'papaya', 'coconut']
Length: 3, dtype: str
--------------------------------------------------
```

In [ ]:
```
### Task 4: Splitting Data into Train and Test Sets
The dataset is split into feature variables (X) and target variable (y), then divided into 80% train and 20% test.
```

In [22]:
```
y = df['label']
x = df.drop(['label'], axis=1)
print('Shape of x', x.shape)
print('Shape of x=y', y.shape)
```

```
Shape of x (2200, 7)
Shape of x=y (2200,)
```

In [23]:
```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)
print('The shape of x train', x_train.shape)
print('The shape of x test', x_test.shape)
print('The shape of y train', x_train.shape)
print('The shape of y test', x_test.shape)
```

```
The shape of x train (1760, 7)
The shape of x test (440, 7)
The shape of y train (1760, 7)
The shape of y test (440, 7)
```

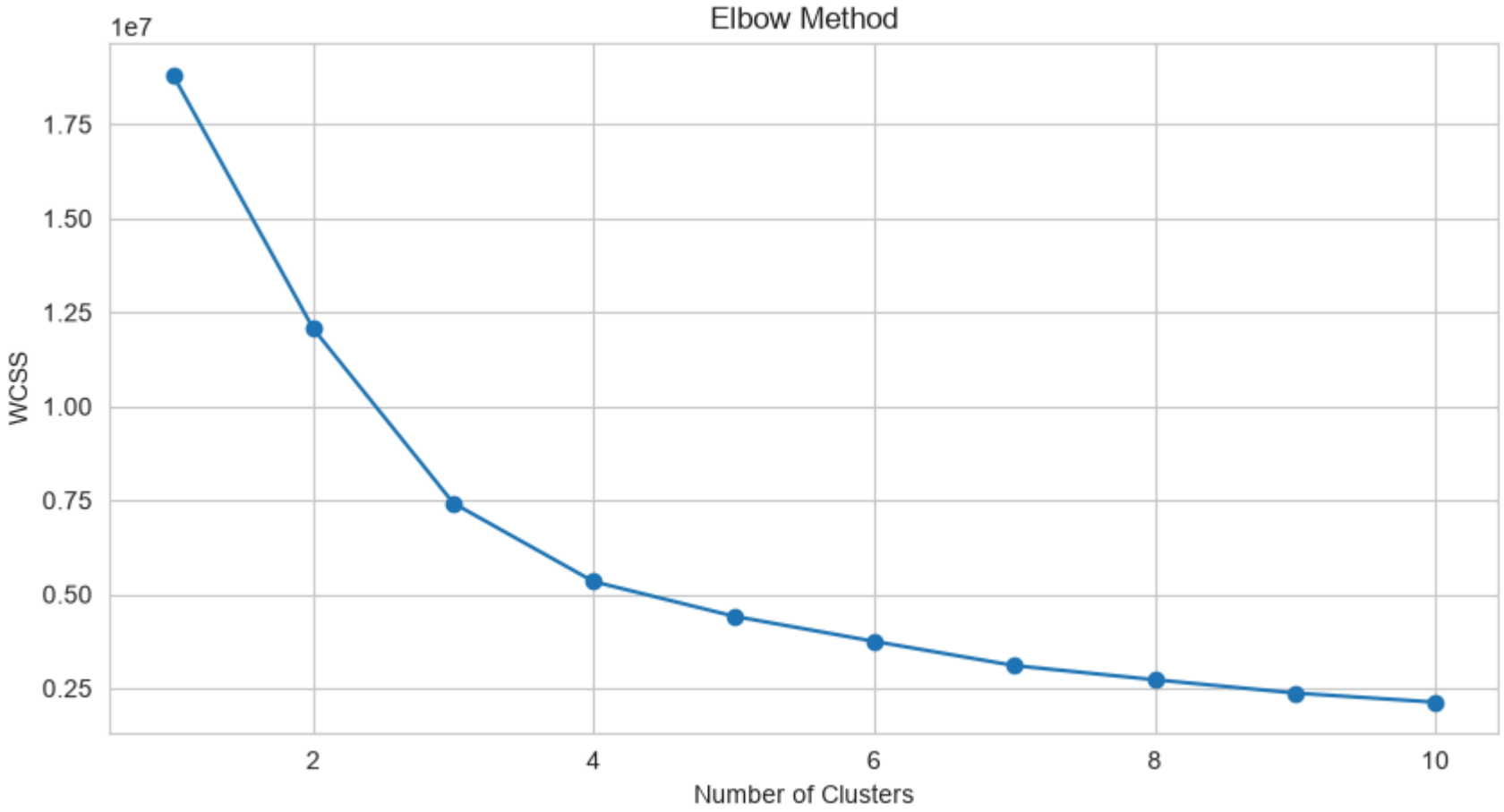
In [ ]:
```
---
## Epic 4: Model Building
---
```

In [24]:
```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
```

In [25]:
```
X = df.drop("label", axis=1)
```

In [26]:
```
plt.rcParams['figure.figsize'] = (10,5)

wcss = []

for i in range(1,11):

    km = KMeans(
        n_clusters=i,
        init='k-means++',
        max_iter=300,
        n_init=10,
        random_state=0
    )

    km.fit(X)
    wcss.append(km.inertia_)
```

In [27]:
```
plt.plot(range(1,11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()
```



In [28]:
```
km = KMeans(
    n_clusters=4,
    init='k-means++',
    max_iter=300,
    n_init=10,
    random_state=0
)

y_means = km.fit_predict(X)
```

In [29]:
```
a = df['label']

y_means = pd.DataFrame(y_means)

z = pd.concat([y_means, a], axis=1)

z = z.rename(columns={0:'cluster'})
```

In [30]:
```
print("Let's check the results after applying the K-Means Clustering Analysis\n")

print("Crops in First Cluster:")
print(z[z['cluster']==0]['label'].unique())

print("-----------------------------------------------")

print("Crops in Second Cluster:")
print(z[z['cluster']==1]['label'].unique())

print("-----------------------------------------------")

print("Crops in Third Cluster:")
print(z[z['cluster']==2]['label'].unique())

print("-----------------------------------------------")

print("Crops in Fourth Cluster:")
print(z[z['cluster']==3]['label'].unique())
```

```
Let's check the results after applying the K-Means Clustering Analysis

Crops in First Cluster:
<ArrowStringArray>
['grapes', 'apple']
Length: 2, dtype: str
-----------------------------------------------
Crops in Second Cluster:
<ArrowStringArray>
[      'maize',    'chickpea', 'kidneybeans',  'pigeonpeas',   'mothbeans',
    'mungbean',   'blackgram',      'lentil', 'pomegranate',       'mango',
      'orange',      'papaya',     'coconut']
Length: 13, dtype: str
-----------------------------------------------
Crops in Third Cluster:
<ArrowStringArray>
['maize', 'banana', 'watermelon', 'muskmelon', 'papaya', 'cotton', 'coffee']
Length: 7, dtype: str
-----------------------------------------------
Crops in Fourth Cluster:
<ArrowStringArray>
['rice', 'pigeonpeas', 'papaya', 'coconut', 'jute', 'coffee']
Length: 6, dtype: str
```

In [ ]:
```
### Task 2: Logistic Regression
Logistic Regression is trained using `fit()` and predictions are generated using `predict()`.
```

In [31]:
```
X = df.drop('label', axis=1)
y = df['label']
```

In [32]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

In [33]:
```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
```

In [34]:
```
model.fit(X_train, y_train)
```

```
C:\Users\HP-PC\AppData\Roaming\Python\Python314\site-packages\sklearn\linear_model\_logistic.py:599: ConvergenceWarning: lbfgs failed to con
verge after 1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

Out[34]:

▾ LogisticRegression ⓘ ?

▸ Parameters

▸ Fitted attributes

In [ ]:
```
### Task 3: Evaluating Model Performance and Saving the Best Model
Model is evaluated using classification metrics. Best model is saved using Pickle.
```

In [39]:
```
# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Train the model
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Classification Report
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

       apple       1.00      1.00      1.00        23
      banana       1.00      1.00      1.00        21
   blackgram       1.00      1.00      1.00        20
    chickpea       1.00      1.00      1.00        26
     coconut       1.00      1.00      1.00        27
      coffee       1.00      1.00      1.00        17
      cotton       1.00      1.00      1.00        17
      grapes       1.00      1.00      1.00        14
        jute       0.92      1.00      0.96        23
 kidneybeans       1.00      1.00      1.00        20
      lentil       0.92      1.00      0.96        11
       maize       1.00      1.00      1.00        21
       mango       1.00      1.00      1.00        19
   mothbeans       1.00      0.96      0.98        24
    mungbean       1.00      1.00      1.00        19
   muskmelon       1.00      1.00      1.00        17
      orange       1.00      1.00      1.00        14
      papaya       1.00      1.00      1.00        23
  pigeonpeas       1.00      1.00      1.00        23
 pomegranate       1.00      1.00      1.00        23
        rice       1.00      0.89      0.94        19
  watermelon       1.00      1.00      1.00        19

    accuracy                           0.99       440
   macro avg       0.99      0.99      0.99       440
weighted avg       0.99      0.99      0.99       440
```

In [40]:

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print('Model Accuracy:', model.score(x_test, y_test))
```

```
Model Accuracy: 0.03409090909090909
```

In [41]:
```
# Save the best model using Pickle
pickle.dump(model, open('model.pkl', 'wb'))
print('Model saved as model.pkl successfully!')
```

```
Model saved as model.pkl successfully!
```

In [ ]:
```
---
## Epic 5: Application Building
---
```

In [ ]:
```
Task 1: Building HTML Pages

Four HTML pages are created to provide an interactive and user-friendly interface for the OptiCrop web application:

index.html – Home page introducing the OptiCrop system with navigation links and project overview.
login.html – User login page that allows registered users to securely access the crop recommendation system.
register.html – Registration page for creating a new user account with secure password storage.
predict.html (or the name of your prediction page, e.g., crop_prediction.html) – Crop recommendation page containing input fields for Nitro

All HTML pages are connected through Flask's url_for() function for seamless navigation. The templates are styled using HTML and CSS to pro
```

In [ ]:
```
### Task 2: Build Python Backend Code
Flask backend (`app.py`) is created with routes for Home, About, FindYourCrop, and Predict. The app reads the port from the `PORT` environm
```

In [ ]:
```
import pickle
import sqlite3
import numpy as np

from flask import Flask, render_template, request, redirect, url_for, session, flash
from werkzeug.security import generate_password_hash, check_password_hash

from utils.predictor import CropPredictor

app = Flask(__name__)
app.secret_key = "smart_agriculture_secret_key"

# ----------------------------
# Database
# ----------------------------
DATABASE = "database.db"

def get_db_connection():
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    return conn
```

```python
# ----------------------------
# ML Model
# ----------------------------
model = pickle.load(open("crop_model.pkl", "rb"))
predictor = CropPredictor()

# ----------------------------
# Home (redirect to login)
# ----------------------------
@app.route("/")
def home():
    return render_template("index.html")

# ----------------------------
# Register
# ----------------------------
@app.route("/register", methods=["GET", "POST"])
def register():

    if request.method == "POST":

        fullname = request.form["fullname"]
        email = request.form["email"]
        password = request.form["password"]

        conn = get_db_connection()

        user = conn.execute(
            "SELECT * FROM users WHERE email=?",
            (email,)
        ).fetchone()

        if user:
            flash("Email already registered!", "danger")
            conn.close()
            return redirect(url_for("register"))

        hashed_password = generate_password_hash(password)

        conn.execute(
            "INSERT INTO users(fullname, email, password) VALUES (?, ?, ?)",
            (fullname, email, hashed_password)
        )

        conn.commit()
```

```
        conn.close()

        flash("Registration Successful! Please Login.", "success")
        return redirect(url_for("login"))

    return render_template("register.html")

# ------------------------------
# Login
# ------------------------------
@app.route("/login", methods=["GET", "POST"])
def login():

    if request.method == "POST":

        email = request.form["email"]
        password = request.form["password"]

        conn = get_db_connection()

        user = conn.execute(
            "SELECT * FROM users WHERE email=?",
            (email,)
        ).fetchone()

        conn.close()

        if user and check_password_hash(user["password"], password):

            session["user_id"] = user["id"]
            session["fullname"] = user["fullname"]

            flash("Login Successful!", "success")
            return redirect(url_for("dashboard"))

        flash("Invalid Email or Password", "danger")

    return render_template("login.html")

# ------------------------------
# Dashboard
# ------------------------------
@app.route("/dashboard")
def dashboard():

    if "user_id" not in session:
```

```
        return redirect(url_for("login"))

    return render_template(
        "dashboard.html",
        name=session["fullname"]
    )

# ----------------------------
# Prediction
# ----------------------------
@app.route("/predict", methods=["POST"])
def predict_route():

    if "user_id" not in session:
        return redirect(url_for("login"))

    try:
        data = request.form

        # Convert form inputs into numeric array
        features = [float(data[key]) for key in data.keys()]
        final_features = np.array([features])

        # Use your ML model directly
        prediction = model.predict(final_features)[0]

        return render_template(
            "dashboard.html",
            name=session["fullname"],
            prediction=prediction
        )

    except Exception as e:
        flash(f"Prediction Error: {str(e)}", "danger")
        return redirect(url_for("dashboard"))

# ----------------------------
# Logout
# ----------------------------
@app.route("/logout")
def logout():

    session.clear()
    flash("Logged Out Successfully!", "success")
    return redirect(url_for("login"))
```

```
# ----------------------------
# Run App
# ----------------------------
if __name__ == "__main__":
    app.run(debug=True)
```

In [ ]:

```
##Task 3: Run the Application

Steps to Run the OptiCrop Flask Application:

Open the OptiCrop project folder in Visual Studio Code.
Ensure the following files are present:
app.py
crop_model.pkl
database.py
database.db
requirements.txt
templates/ folder containing the HTML pages
Install the required dependencies:
pip install -r requirements.txt

Or install them manually:

pip install flask pandas numpy scikit-learn werkzeug
Run the Flask application:
python app.py
Open the server URL displayed in the terminal:
http://127.0.0.1:5000
Pages:
Home Page – Displays the OptiCrop homepage with navigation options.
Login Page – Allows registered users to log in securely.
Register Page – Enables new users to create an account.
Crop Recommendation Page – Users enter soil and environmental parameters (N, P, K, Temperature, Humidity, pH, Rainfall) and click Predict t
Output:

Recommended Crop: Rice (or another suitable crop based on the input values provided by the user).
```